

Java Programming

History of Java

- Developed by Sun Microsystems of USA in 1991
- Originally called Oak by James Gosling inventor of Java Language
- It was designed for development of software for consumer electronic devices like TV, VCR etc.

Characteristics of Java

- Java is simple
- Java is object-oriented
- Java is distributed
- Java is interpreted
- Java is robust
- Java is secure
- Java is architecture-neutral
- Java is portable
- Java's performance
- Java is multithreaded
- Java is dynamic

Simple Application

Ex 1.1 ://This application program prints Welcome

//to Java!

`package chapter1;` (A **package** in Java is a mechanism used to group related classes, interfaces, enumerations, and sub-packages together. It functions exactly like a folder on a computer file system, mapping directly to directory structures to keep the code modular and organized.)

```
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Java Program Consists of

- Comments
- Reserved words
- Modifiers
- Statements
- Blocks
- Classes
- Methods
- The main method
- The exit method

Comments

In Java, comments are preceded by two slashes (`//`) in a line, or enclosed between `/*` and `*/` in one or multiple lines. When the compiler sees `//`, it ignores all text after `//` in the same line. When it sees `/*`, it scans for the next `*/` and ignores any text between `/*` and `*/`.

Reserved words

Reserved words or *keywords* are words that have a specific meaning to the compiler and cannot be used for other purposes in the program. For example, when the compiler sees the word class, it understands that the word after class is the name for the class. Other reserved words in Example 1.1 are public, static, and void.

Modifiers

Java uses certain reserved words called modifiers that specify the properties of the data, methods, and classes and how they can be used. Examples of modifiers are public and static. Other modifiers are private, final, abstract, and protected. A public datum, method, or class can be accessed by other programs. A private datum or method cannot be accessed by other programs.

Statement

A statement represents an action or a sequence of actions. The statement `System.out.println("Welcome to Java!");` in the program in Example 1.1 is a statement to display the greeting "Welcome to Java!" Every statement in Java ends with a semicolon (;).

Blocks

A pair of braces in a program forms a block that groups components of a program.

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Class block

Method block

Classes

The *class* is the essential Java construct. A class is a template or blueprint for objects.

A java program is defined by using one or more classes.

Methods

- What is System.out.println? It is a *method*: a collection of statements that performs a sequence of operations to display a message on the console.
- It can be used even without fully understanding the details of how it works.
- It is used by invoking a statement with a string argument. The string argument is enclosed within parentheses. In this case, the argument is "Welcome to Java!"
- You can call the same println method with a different argument to print a different message.

The main method provides the control of program flow. The Java interpreter executes the application by invoking the main method.

The main method looks like this:

```
public static void main(String[] args) {  
    // Statements;  
}
```

The exit method

Use Exit to terminate the program and stop all threads.

When your program starts, a thread is spawned to run the program. To terminate the thread, you have to invoke the exit method.

Primitive data types and operations

- Introduce Programming with an Example
- Identifiers, Variables, and Constants
- Primitive Data Types
 - byte, short, int, long, float, double, char, boolean
- Expressions
- Operators, Precedence, Associativity, Operand Evaluation Order: ++, --, *, /, %, +=, -=, *=, /=, %=, ^, &, |, +, -,
- Getting Input from Input Dialog Boxes
- Case Studies (Computing Mortgage, and Computing Changes)
- Style and Documentation Guidelines
- Syntax Errors, Runtime Errors, and Logic Errors

Identifiers

- An identifier is a sequence of characters that consist of letters, digits, underscores (`_`), and dollar signs (`$`).
- An identifier must start with a letter, an underscore (`_`), or a dollar sign (`$`). It cannot start with a digit.
- An identifier cannot be a reserved word.
- An identifier cannot be `true`, `false`, or `null`.
- An identifier can be of any length.

Variables

```
// Compute the first area
radius = 1.0;
area = radius*radius*3.14159;
System.out.println("The area is " + area + " for radius "+radius);

// Compute the second area
radius = 2.0;
area = radius*radius*3.14159;
System.out.println("The area is " + area + " for radius "+radius);
```

Declaring Variables

```
int x;           // Declare x to be an
                  // integer variable;

double radius;  // Declare radius to
                  // be a double variable;

char a;         // Declare a to be a
                  // character variable;
```

Numeric Data types

byte	8 bits
------	--------

short	16 bits
-------	---------

int	32 bits
-----	---------

long	64 bits
------	---------

float	32 bits
-------	---------

double	64 bits
--------	---------

Assignment Statements

```
x = 1;           // Assign 1 to x;
```

```
radius = 1.0;    // Assign 1.0 to radius;
```

```
a = 'A';         // Assign 'A' to a;
```

Constants

```
final datatype CONSTANTNAME = VALUE;
```

```
final double PI = 3.14159;
```

```
final int SIZE = 3;
```

Operators

$+$, $-$, $*$, $/$, and $\%$

$5/2$ yields an integer 2.

$5.0/2$ yields a double value 2.5

$5 \% 2$ yields 1 (the remainder of the division)

$5.0 \% 2$ is not defined : modulo is defined only for integers.

Addition Program

```
public class addition {  
    public static void main(String[] args) {  
  
        // declare variables  
        int x, y, z;  
  
        // Specify values of x and y  
        x = 2;  
        y = 3;  
  
        z = x + y;  
  
        System.out.println("x has a value of " + x);  
        System.out.println("y has a value of " + y);  
        System.out.println("The sum of x + y is " + z);  
  
        System.exit(0);  
    }  
}
```

Division Program

```
public class division {  
    public static void main ( String[] args) {  
  
        //declare variables  
        int x, y, z ;  
        x = 12;  
        y = 4;  
        z = x / y ;  
  
        System.out.println("x has a value of " + x);  
        System.out.println("y has a value of " + y);  
        System.out.println("x divided by y is " + z);  
  
        System.exit(0);  
    }  
}
```

How to download Java?

- From www.java.sun.com/j2se
- Click on j2se 5.0
- See on your right (popular downloads) ..click on j2se 5.0
- Click on download jdk 5.0 update 3
- Accept the agreement and continue
- Download the version as per your platform

Compile and Run a Java Program

Java source code files (files with a *.java* extension) are compiled into a format called *bytecode* (files with a *.class* extension), which can then be executed by a Java interpreter. Compiled Java code can run on most computers because Java interpreters and runtime environments, known as *Java Virtual Machines (VMs)*, exist for most operating systems, including UNIX, the MACintosh OS, and Windows. Bytecode can also be converted directly into machine language instructions by a just-in-time compiler (JIT).

For Example

- Create a file named say .. addition.java using some editor say wordpad.
- From the command line type the following
 - `javac addition.java` (java code compiled to a bytecode)
- A file called addition.class(bytecode) is created,
- Now type
 `java addition` (bytecode being executed by java interpreter)
And you will get the results

Number Literals

A *literal* is a constant value that appears directly in the program. For example, 34, 1,000,000, and 5.0 are literals in the following statements:

```
int i = 34;
```

```
long l = 1000000;
```

```
double d = 5.0;
```

Integer Literals

An integer literal can be assigned to an integer variable as long as it can fit into the variable. A compilation error would occur if the literal were too large for the variable to hold. For example, the statement `byte b = 1000` would cause a compilation error, because 1000 cannot be stored in a variable of the `byte` type.

An integer literal is assumed to be of the `int` type, whose value is between -2^{31} (-2147483648) to $2^{31}-1$ (2147483647). To denote an integer literal of the `long` type, append it with the letter `L` or `l`. `L` is preferred because `l` (lowercase `L`) can easily be confused with `1` (the digit one).

Floating-Point Literals

Floating-point literals are written with a decimal point. By default, a floating-point literal is treated as a double type value. For example, 5.0 is considered a double value, not a float value. You can make a number a float by appending the letter f or F, and make a number a double by appending the letter d or D. For example, you can use 100.2f or 100.2F for a float number, and 100.2d or 100.2D for a double number.

Scientific Notation

Floating-point literals can also be specified in scientific notation, for example, $1.23456e+2$, same as $1.23456e2$, is equivalent to 123.456, and $1.23456e-2$ is equivalent to 0.0123456. E (or e) represents an exponent and it can be either in lowercase or uppercase.

Arithmetic Expressions

$$\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)$$

is translated to

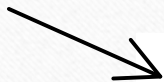
$$\underline{(3+4*x) / 5 - 10 * (y-5) * (a+b+c) / x + 9 * (4/x + (9+x) / y)}$$

Shortcut Assignment Operators

<i>Operator</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	<code>i+=8</code>	<code>i = i+8</code>
<code>-=</code>	<code>f-=8.0</code>	<code>f = f-8.0</code>
<code>*=</code>	<code>i*=8</code>	<code>i = i*8</code>
<code>/=</code>	<code>i/=8</code>	<code>i = i/8</code>
<code>%=</code>	<code>i%=8</code>	<code>i = i%8</code>

Increment and Decrement Operators

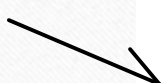
suffix

 `x++;` // Same as `x = x + 1;`

prefix

 `++x;` // Same as `x = x + 1;`

suffix

 `x--;` // Same as `x = x - 1;`

prefix

 `--x;` // Same as `x = x - 1;`

Increment and Decrement Operators, cont.

```
int i=10;  
int newNum = 10*i++;
```

Equivalent to

```
int newNum = 10*i;  
i = i + 1;
```

```
int i=10;  
int newNum = 10*(++i);
```

Equivalent to

```
i = i + 1;  
int newNum = 10*i;
```

Increment and Decrement Operators, cont.

Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read.

Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this:
int k = ++i + i. Its not a good programming practice.

```
public class assignment_operators {
    public static void main(String[] args) {
        // declare variables
        int x = 10;
        int y = 5;
        int z = 3;
        System.out.println("x = "+x+", y = "+y+", z = "+z);
        x++; // x = x+1;
        y += x; // y = y+x;
        z *= x; // z = z*x;
        System.out.println("Now x = "+x+", y = "+y+", z = "+z);
        x--; // x = x-1;
        y *= x; // y = y*x;
        z %= x; // z = z%x;
        System.out.println("And now x = "+x+", y = "+y+", z = "+z);
        System.exit(0);
    }
}
```

```
public class circle_area {
    public static void main(String[] args) {

        // declare variables
        double radius, area;

        // assign radius of the circle
        radius = 3.00;

        area = radius * radius * 3.14159 ;
        System.out.println("The area of the circle of radius " + radius +
" is " + area);
        System.exit(0);
    }
}
```

```
public class circle_area_pi {  
    public static void main(String[] args) {  
  
        final double PI = 3.14159;  
        // declare variables  
        double radius, area;  
  
        // assign radius of the circle  
        radius = 3.00;  
  
        area = radius * radius * PI ;  
        System.out.println("The area of the circle of radius " + radius +  
" is " + area);  
        System.exit(0);  
    }  
}
```

Assignment Expressions and Assignment Statements

`variable op= expression; // Where op is +,
-, *, /, or %`

`++variable;`

`variable++;`

`--variable;`

`variable--;`

Numeric Type Conversion

Consider the following statements:

```
byte i = 100;
```

```
long k = i*3+4;
```

```
double d = i*3.1+k/2;
```

```
int x = k; // (Wrong)
```

```
long k = x; // (fine, implicit casting)
```

Type Casting

- double
- float
- long
- int
- short
- byte

Type Casting, cont.

Implicit casting

```
double d = 3; (type widening)
```

Explicit casting

```
int i = (int)3.0; (type narrowing)
```

Character Data Type

```
char letter = 'A'; (ASCII)
```

```
char numChar = '4'; (ASCII)
```

```
char letter = '\u0041'; (Unicode)
```

```
char numChar = '\u0034'; (Unicode)
```

Special characters

```
char tab = '\t';
```

Unicode Format

<i>Description</i>	<i>Escape Sequence</i>	<i>Unicode</i>
Backspace	\b	\u0008
Tab	\t	\u0009
Linefeed	\n	\u000a
Carriage return	\r	\u000d

Casting between char and Numeric Types

```
int i = 'a'; // Same as int i = (int) 'a';
```

```
char c = 97; // Same as char c = (char) 97;
```

The boolean Type and Operators

```
boolean lightsOn = true;  
boolean lightsOn = false;  
boolean b = (1 > 2);
```

- `&&` (and) `(1 < x) && (x < 100)`
- `||` (or) `(lightsOn) || (isDayTime)`
- `!` (not) `!(isStopped)`

Comparison Operators

Operator *Name*

< less than

<= less than or equal to

> greater than

>= greater than or equal to

== equal to

!= not equal to

Boolean Operators

<i>Operator</i>	<i>Name</i>
!	not
& &	and
	or
^	exclusive or

Operator Precedence

- `var++`, `var--`
- `+`, `-` (Unary plus and minus), `++var`, `--var`
- `(type)` Casting
- `!` (Not)
- `*`, `/`, `%` (Multiplication, division, and modulus)
- `+`, `-` (Binary addition and subtraction)
- `<`, `<=`, `>`, `>=` (Comparison)
- `==`, `!=`; (Equality)
- `&` (Unconditional AND)
- `^` (Exclusive OR)
- `|` (Unconditional OR)
- `&&` (Conditional AND) Short-circuit AND
- `||` (Conditional OR) Short-circuit OR
- `=`, `+=`, `-=`, `*=`, `/=`, `%=` (Assignment operator)

Operator Associativity

When two operators with the same precedence are evaluated, the *associativity* of the operators determines the order of evaluation. All binary operators except assignment operators are *left-associative*.

$a - b + c - d$ is equivalent to $((a - b) + c) - d$

Assignment operators are *right-associative*. Therefore, the expression

$a = b += c = 5$ is equivalent to $a = (b += (c = 5))$

Operand Evaluation Order

The precedence and associativity rules specify the order of the operators, but do not specify the order in which the operands of a binary operator are evaluated. Operands are evaluated from left to right in Java.

The left-hand operand of a binary operator is evaluated before any part of the right-hand operand is evaluated.

Operand Evaluation Order, cont.

If no operands have *side effects* that change the value of a variable, the order of operand evaluation is irrelevant. Interesting cases arise when operands do have a side effect. For example, `x` becomes 1 in the following code, because `a` is evaluated to 0 before `++a` is evaluated to 1.

```
int a = 0; int x = a + (++a);
```

But `x` becomes 2 in the following code, because `++a` is evaluated to 1, then `a` is evaluated to 1.

```
int a = 0; int x = ++a + a;
```

Operator Precedence

How to evaluate

$3 + 4 * 4 > 5 * (4 + 3) - ++i$

Lets parenthesize

$(3 + (4 * 4)) > ((5 * (4 + 3)) - (++i))$

This evaluates to

$19 > (35 - (++i))$

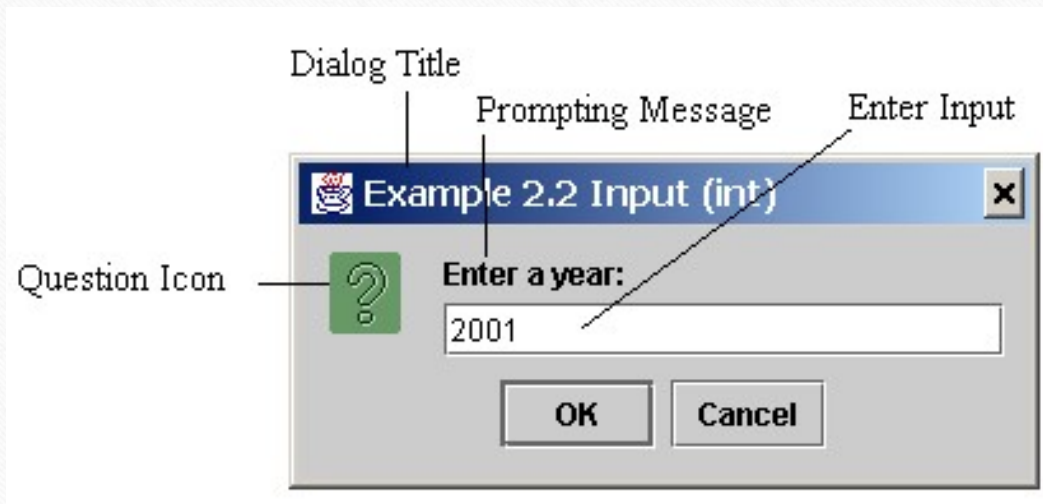
This evaluates to true if the value of i just before this expression is > 15

Getting Input from Input

Dialog Boxes

```
String string = JOptionPane.showInputDialog(  
    null, "Prompt Message", "Dialog Title",  
    JOptionPane.QUESTION_MESSAGE));
```

where x is a string for the prompting message and y is a string for the title of the input dialog box.



Converting Strings to Integers

The input returned from the input dialog box is a string. If you enter a numeric value such as 123, it returns "123". To obtain the input as a number, you have to convert a string into a number.

To convert a string into an int value, you can use the static parseInt method in the Integer class as follows:

```
int intValue = Integer.parseInt(intString);
```

where intString is a numeric string such as "123".

Converting Strings to Doubles

To convert a string into a double value, you can use the static parseDouble method in the Double class as follows:

```
double doubleValue  
=Double.parseDouble(doubleString);
```

where doubleString is a numeric string such as "123.45".

Programming Errors

- Syntax Errors
 - Detected by the compiler
- Runtime Errors
 - Causes the program to abort
- Logic Errors
 - Produces incorrect result

Compilation Errors

```
public class ShowSyntaxErrors {  
    public static void main(String[] args)  
    {  
        i = 30  
        System.out.println(i+4);  
    }  
}
```

Runtime Errors

```
public class ShowRuntimeErrors {  
    public static void main(String[] args)  
    {  
        int i = 1 / 0;  
    }  
}
```

Logic Errors

```
public class ShowLogicErrors {  
    // Determine if a number is between 1 and 100 inclusively  
    public static void main(String[] args) {  
        // Prompt the user to enter a number  
        String input = JOptionPane.showInputDialog(null,  
            "Please enter an integer:",  
            "ShowLogicErrors", JOptionPane.QUESTION_MESSAGE);  
        int number = Integer.parseInt(input);  
  
        // Display the result  
        System.out.println("The number is between 1 and 100, " +  
            "inclusively? " + ((1 < number) && (number < 100)));  
        System.exit(0);  
    }  
}
```

Prog for area of circle:

```
import java.util.Scanner;// Define the Circle class to represent the object
```

```
class Circle { // Instance variable to store the radius
```

```
private double radius; // Constructor to initialize the radius
```

```
public Circle(double radius)
```

```
{    this.radius = radius; } // Method to calculate and return the area of  
the circle
```

```
public double calculateArea() { // Math.PI provides the precise value of  
 $\pi$  (3.14159...)
```

```
return Math.PI * radius * radius; } }//
```

```
public class Main {  
    public static void main(String[] args)  
    {  
        // Create a Scanner object to read user input from the console  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Enter the radius of the circle: ");  
        double inputRadius = scanner.nextDouble(); // Create an object of the Circle class using  
        the 'new' keyword  
        Circle myCircle = new Circle(inputRadius); // Call the method using the object to get the  
        area  
        double area = myCircle.calculateArea();    // Display the calculated result  
        System.out.printf("The area of the circle is: %.2f\n", area);    // Close the scanner resource  
        scanner.close();    }}
```

Program for even odd number

```
import java.util.Scanner; // Import the Scanner class to read user input
public class EvenOddProgram
{
    public static void main(String[] args)
    { // Create a Scanner object to read input from the console
        Scanner scanner = new Scanner(System.in); // Prompt the user to enter an integer
        System.out.print("Enter a number: ");
        int number = scanner.nextInt(); // Check if the number is divisible by 2
        if (number % 2 == 0)
        { System.out.println(number + " is an EVEN number."); }
        Else
        { System.out.println(number + " is an ODD number."); }
        scanner.close(); } } // Close the scanner to avoid memory leaks
```

Program based on display student details

```
class Student {
```

```
String name; int grade; // Constructor to assign unique properties to each  
object
```

```
Student(String studentName, int studentGrade)
```

```
{
```

```
name = studentName;
```

```
grade = studentGrade; }
```

```
void displayInfo()
```

```
{ System.out.println("Student: " + name + " | Grade: " + grade); } }
```

```
public class Main
{
    public static void main(String[] args)
    { // Creating two completely distinct objects
        Student student1 = new Student("Alice", 11);
        Student student2 = new Student("Bob", 12); // Interacting with each object independently
        student1.displayInfo();
        student2.displayInfo(); } }
```